KISHKINDA UNIVERSITY

Advancing Knowledge Transforming Lives

Department of Computer Science & Engineering

DIGITAL SYSTEM DESIGN AND COMPUTER ORGANIZATION (23CS32)

(Academic Year - 2023-24)

Semester – III

DIGITAL LABORATORY

List of Experiments

1. a) Realize 3-varianle and 4-variable Boolean expressions, simplify using K-map and Implementusing Basic Gates.

   b) Simulate and verify the working of above expressions using VHDL.

2. a) Design and Implement Half Adder and Full Adder using Basic Gates.

   b) Simulate and verify the working of Half Adder and Full Adder using VHDL.

3. a) Given a 4-variable logic expression, simplify it using Entered Variable Map and realize thesimplified logic expression using 8:1 multiplexer.

   b) Simulate and verify the working of 8:1 multiplexer using VHDL.

4. a) Design and Implement the Binary to Gray Code converter using Basic Gates.

   b) Simulate and verify the working of Binary to Gray Code converter using VHDL.

5. Design and Implement the Truth Table of 3-bit Parity Generator and 4-bit Parity Checker with anEven parity bit using Basic Gates.

6. a) Realize a J-K Master / Slave Flip-Flop using NAND gates and verify its truth table.

   b) Simulate and verify the working of D Flip-Flop with positive edge triggering using VHDL.

7. a) Realize Design and Implement a MOD-n (n<8) Synchronous Up Counter using J-K Flip-flopICs.

   b) Simulate and verify the working of mod-8 up counter using VHDL.

8. a) Design and Implement an Asynchronous Counter using Decade counter IC to Count Up from 0to n (n<=9) and Demonstrate on 7-Segment Display (using IC7447).

   b) Simulate and verify the working of Switched tail Counter using VHDL.

Introduction to Logic Gates

Apparatus Required

SL. No.	Component	Specification
1	AND GATE	IC 7408
2	OR GATE	IC 7432
3	NOT GATE	IC 7404
4	NAND	IC 7400
5	3 Input NAND	IC7410
6	IC TRAINER KIT	-
7	PATCH CORDS	-

Theory

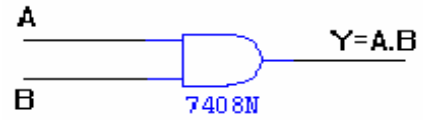
Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on certain **logic**. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.

AND Gate

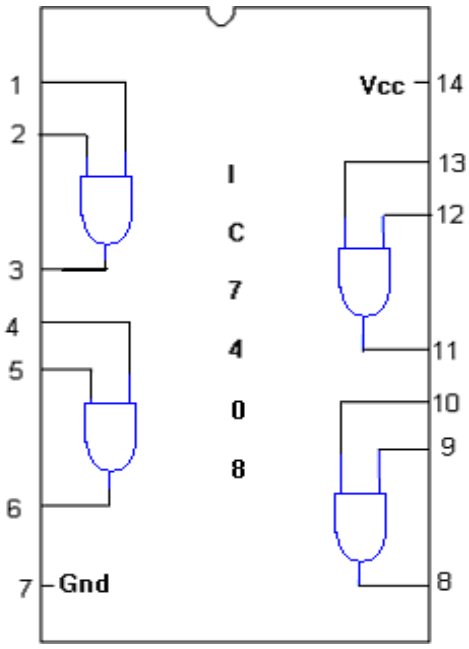
An AND gate has a single output and two or more inputs.

1. When all of the inputs are 1, the output of this gate is 1.
2. The AND gate's Boolean logic is Y=A.B if there are two inputs A and B.

SYMBOL:



PIN DIAGRAM:



TRUTH TABLE

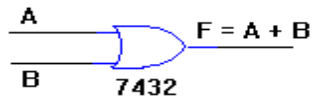
A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

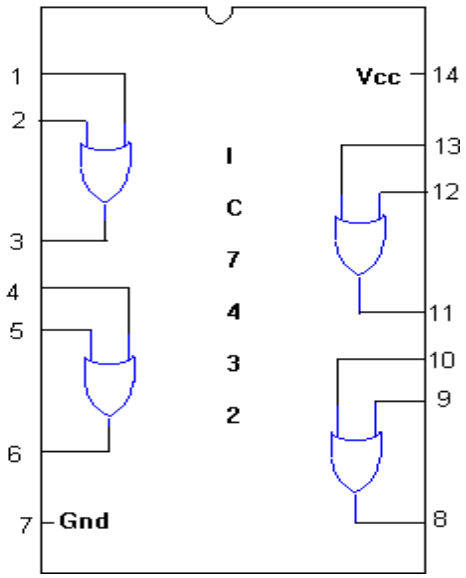
An **OR gate** is a logic gate that performs logical OR operation. A logical OR operation has a high output (1) if one or both the inputs to the gate are high (1). If neither input is high, a low output (0) results. Just like an AND gate, an OR gate may have any number of input probes but only one output probe.

The function of a logical OR gate effectively finds the maximum between two binary digits, just as the complementary AND function finds the minimum.

SYMBOL :



PIN DIAGRAM :



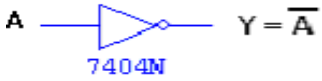
TRUTH TABLE

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

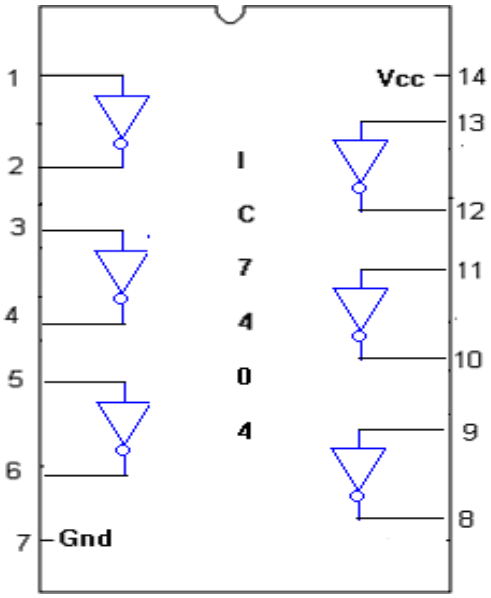
NOT Gate

A **NOT gate** is a logic gate that inverts the digital input signal. For this reason, a NOT gate is sometimes is referred to as an inverter. A NOT gate always has high (logical 1) output when its input is low (logical 0). Conversely, a **logical NOT gate** always has low (logical 0) output when the input is high (logical 1).

SYMBOL:



PIN DIAGRAM:



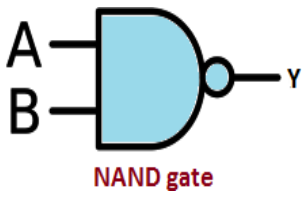
TRUTH TABLE :

A	$\overline{A}$
0	1
1	0

NAND Gate

The NAND (Not – AND) gate has an output that is normally at logic level "1" and only goes "LOW" to logic level "0" when **ALL** of its inputs are at logic level "1". The **Logic NAND Gate** is the reverse or "Complementary" form of the AND gate we have seen previously.

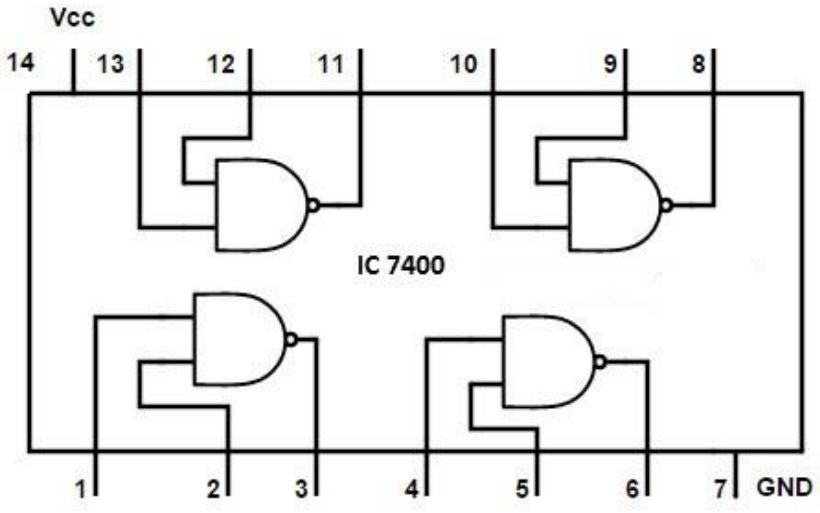
Symbol



Truth Table

Input		Output
A	B	Y= $\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0

Pin Diagram



Experiment – 1

a) Realize 3-varianle and 4-variable Boolean expressions, simplify using K-map and Implementusing Basic Gates.

Apparatus Required

SL. No.	Component	Specification
1	AND GATE	IC 7408
2	OR GATE	IC 7432
3	NOT GATE	IC 7404
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

Karnaugh maps are used to simplify real-world logic requirements so that they can be implemented using a minimum number of logic gates. A sum-of-products expression (SOP) can always be implementedusing AND gates feeding into an OR gate, and a product-of-sums expression (POS) leads to OR gates feeding an AND gate.

The POS expression gives a complement of the function (if F is the function so its complement will be F'). Karnaugh maps can also be used to simplify logic expressions in software design. Boolean conditions, as used for example in conditional statements, can get very complicated, which makes the codedifficult to read and to maintain. Once minimized, canonical sum-of-products and product-of-sums expressions can be implemented directly using AND and OR logic operators.

Applications of Karnaugh's Map

- They are used in design and implementation of Digital circuits
- K-maps are useful for detecting and eliminating race condition

Procedure

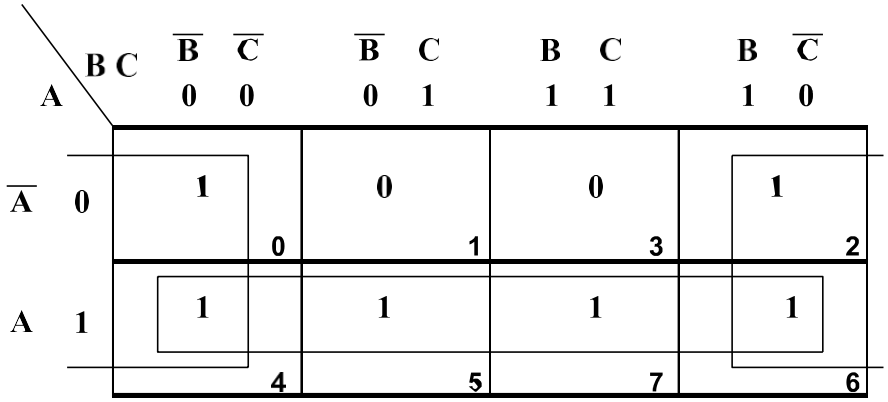
1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, and verify the outputs intrainer kit.
5. Switch off the power supply and remove the connections.

3 – Variable Boolean Expression

F(A, B, C) = ∑m (0, 2, 4, 5, 6, 7)

$$Y = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}\,B\,\overline{C} + A\,\overline{B}\,\overline{C} + A\,\overline{B}\,C + A\,B\,\overline{C} + A\,B\,C$$

K – Map

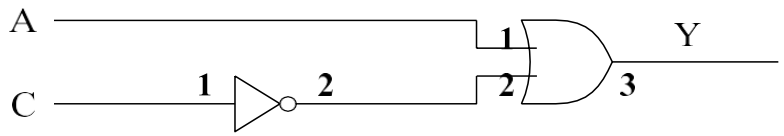


Truth table

A	C	Y
0	0	1
0	1	0
1	0	1
1	1	1

$$Y = A + \overline{C}$$

Circuit Diagram

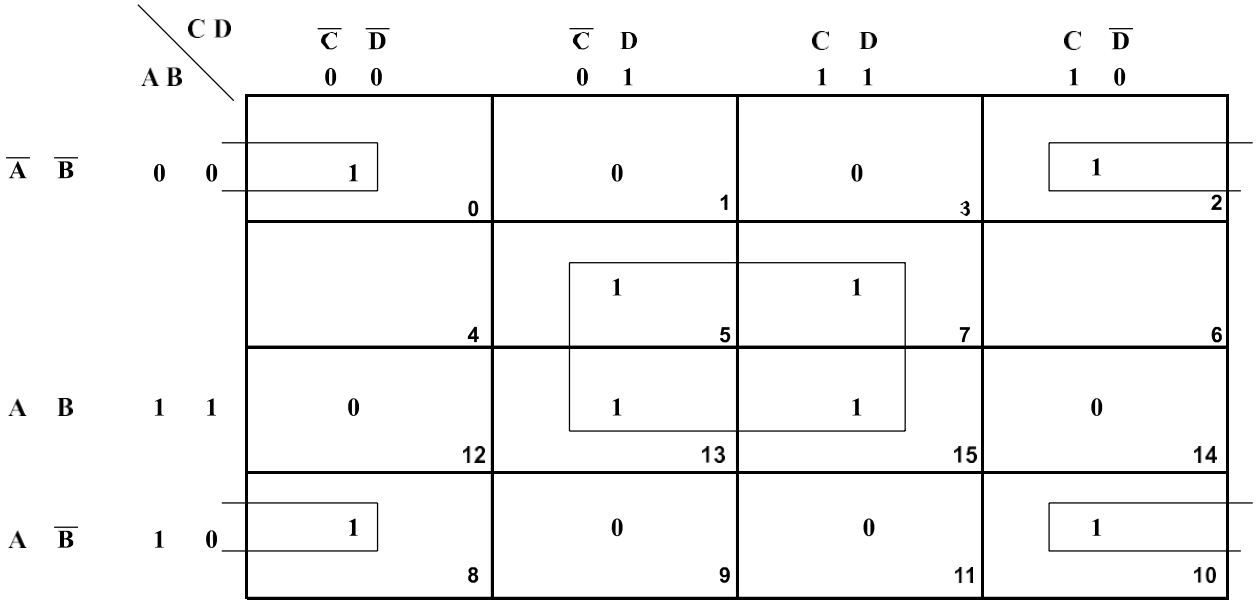


4 – Variable Boolean Expression

F(A, B, C, D) = ∑m (0, 2, 5, 7, 8, 10, 13, 15)

$$Y = \overline{A}\,\overline{B}\,\overline{C}\,\overline{D} + \overline{A}\,\overline{B}\,C\,\overline{D} + \overline{A}\,B\,\overline{C}\,D + \overline{A}\,B\,C\,D + A\,\overline{B}\,\overline{C}\,\overline{D} + A\,\overline{B}\,C\,\overline{D} + A\,B\,\overline{C}\,D + A\,B\,C\,D$$

K – Map

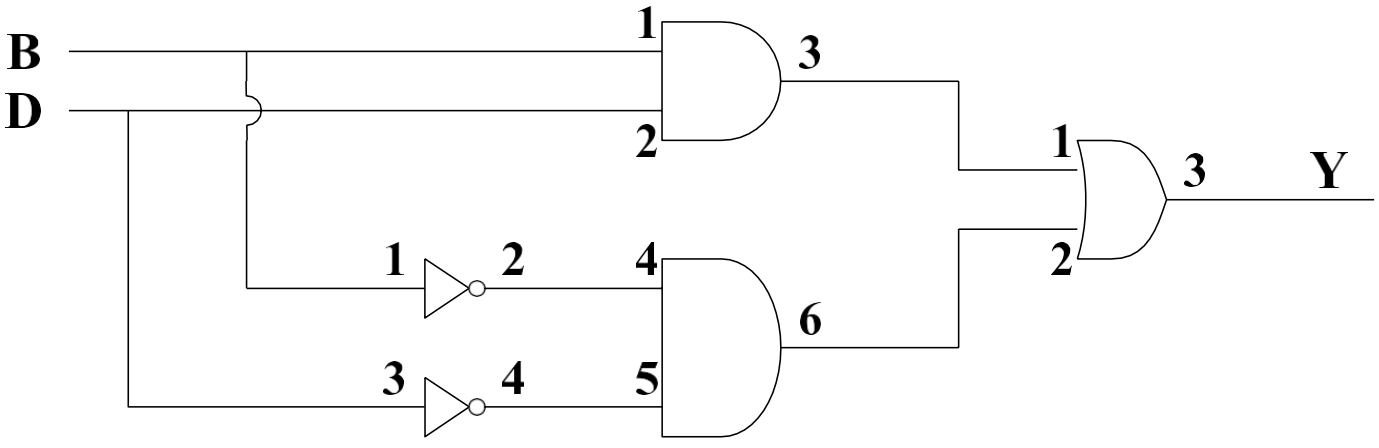


$$Y = \overline{B}\overline{D} + B\,D$$

Truth Table

INPUTS						OUTPUT
B	D	B'	D'	B'D'	BD	B'D' + BD
0	0	1	1	1	0	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	0	1	1

Circuit Diagram



Result

3 – Variable and 4 – Variable Boolean expressions are simplified and verified using Basic Gates.

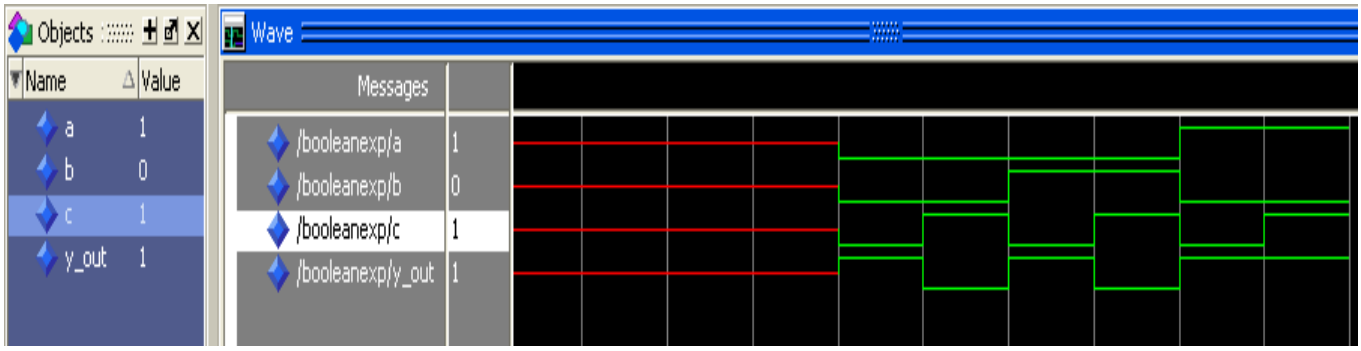
b) Simulate and verify the working of above expressions using VHDL.

3 – Variable Boolean Expression

$\mathbf{Y = A + \overline{C}}$

```
entity three_variable_booleanExp
   isPort ( a,b,c : in  STD_LOGIC;
         y_out : out  STD_LOGIC);
end three_variable_booleanExp;

architecture Behavioral of three_variable_booleanExp is
begin
          y_out<=(a or (not(c)));
end Behavioral;
```

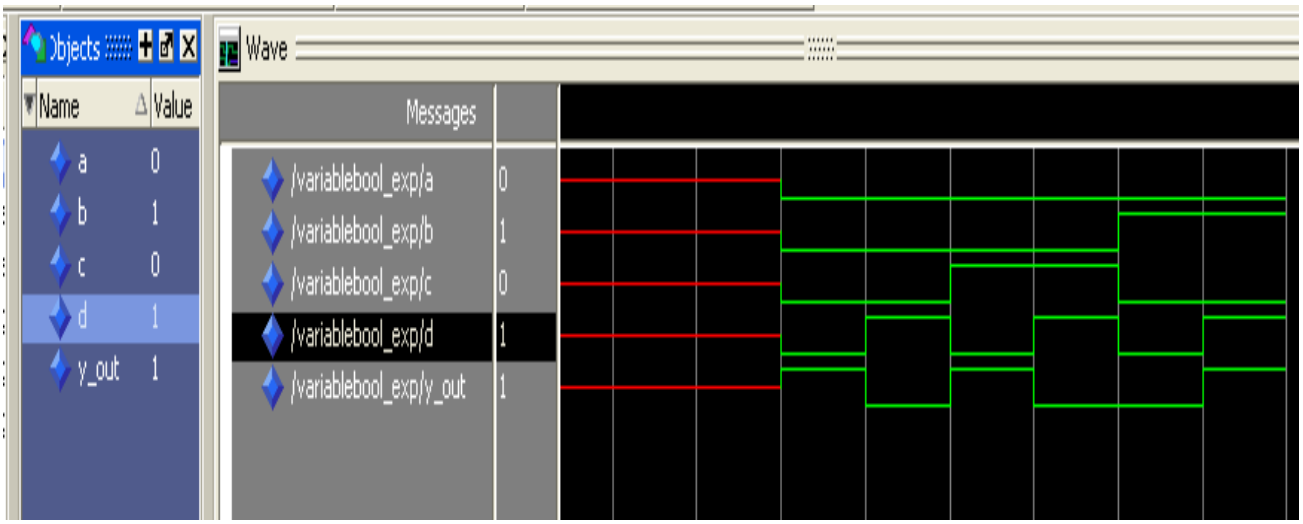


4– Variable Boolean Expression

$\boldsymbol{Y = \overline{B}\overline{D} + B D}$

```
entity Four_VariableBool_Exp is
   Port ( a,b,c,d : in  STD_LOGIC;
         y_out : out  STD_LOGIC);
end Four_VariableBool_Exp;

architecture Behavioral of Four_VariableBool_Exp is
begin
y_out<=(((not(b)) and (not(d)))or(b and d));
   end Behavioral;
```


Objects
Name
Value
a
0
b
1
c
0
d
1
y_out
1
Wave
Messages
/variablebool_exp/a
0
/variablebool_exp/b
1
/variablebool_exp/c
0
/variablebool_exp/d
1
/variablebool_exp/y_out
1

Experiment – 2

a) Design and Implement Half Adder and Full Adder using Basic Gates.

Apparatus Required

SL. No.	Component	Specification
1	AND GATE	IC 7408
2	OR GATE	IC 7432
3	NOT GATE	IC 7404
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

Half Adder

A half adder has two inputs for the two bits to be added and two outputs one from the sum ' S' and other from the carry ' c' into the higher adder position. Above circuit is called as a carry signal from the addition of the less significant bits sum from the X-OR Gate the carry out from the AND gate.

Full Adder

A full adder is a combinational circuit that forms the arithmetic sum of input; it consists of three inputs and two outputs. A full adder is useful to add three bits at a time but a half adder cannot do so. In fulladder sum output will be taken from X-OR Gate, carry output will be taken from OR Gate.

Applications

- ❖ They are used computers, calculators and various digital measuring instruments
- ❖ They are used in digital processing devices

Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the

outputs of thetrainerkit.

Switch off the power supply and remove the connections.

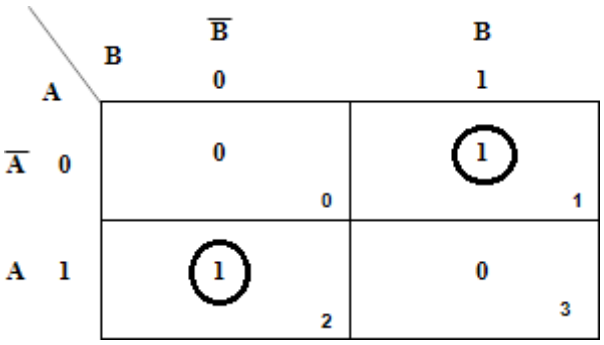
Half Adder

Truth Table

Cell No.	Inputs		Outputs	
	A	B	Sum	Carry
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1

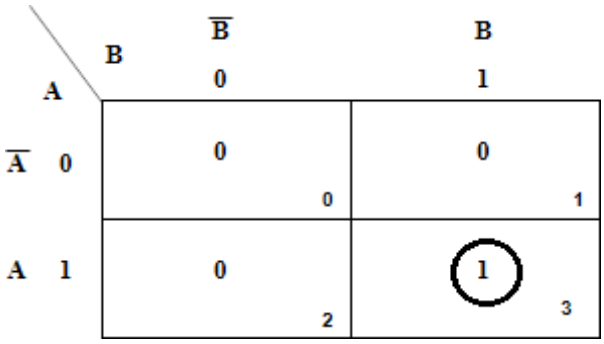
K – Map

Sum



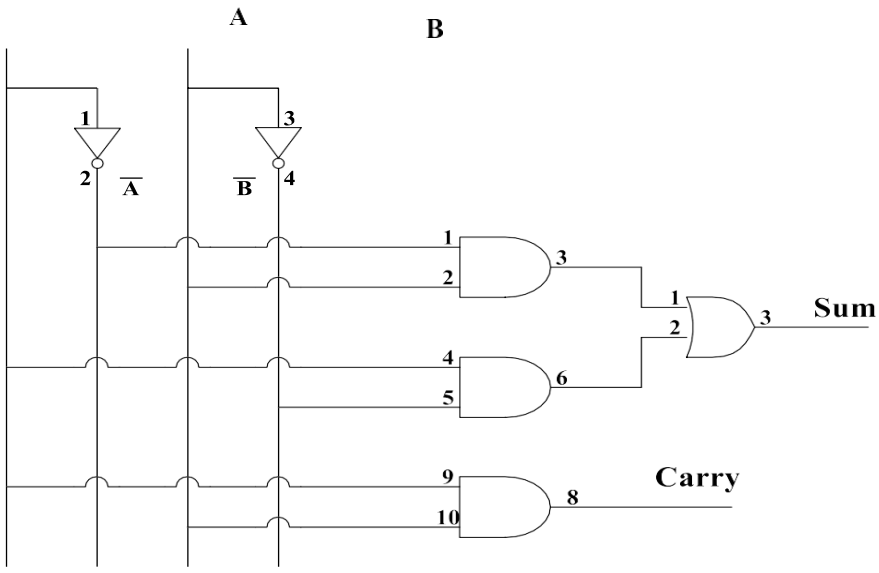
Sum = $\overline{A}B + A\overline{B}$

Carry



$Carry = A\,B$

Circuit Diagram

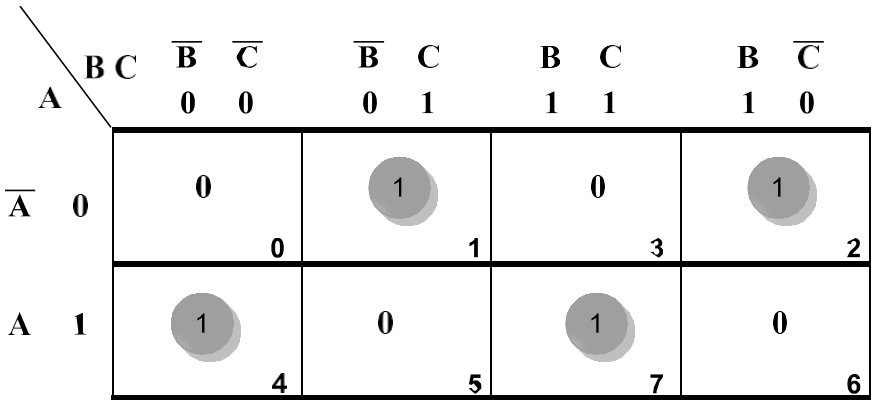


Full Adder

Truth Table

Cell No.	Inputs			Outputs	
	A	B	C	Sum	Carry
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

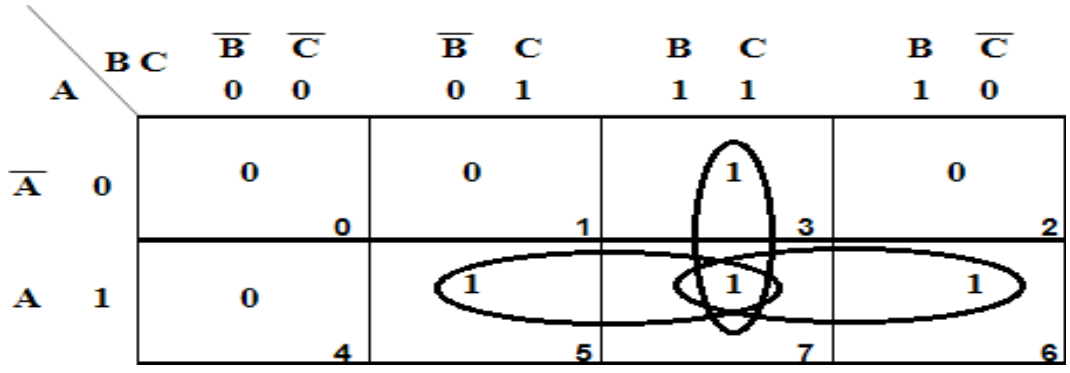
K – Map for Sum:



$$Sum = \bar{A}B\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + ABC$$

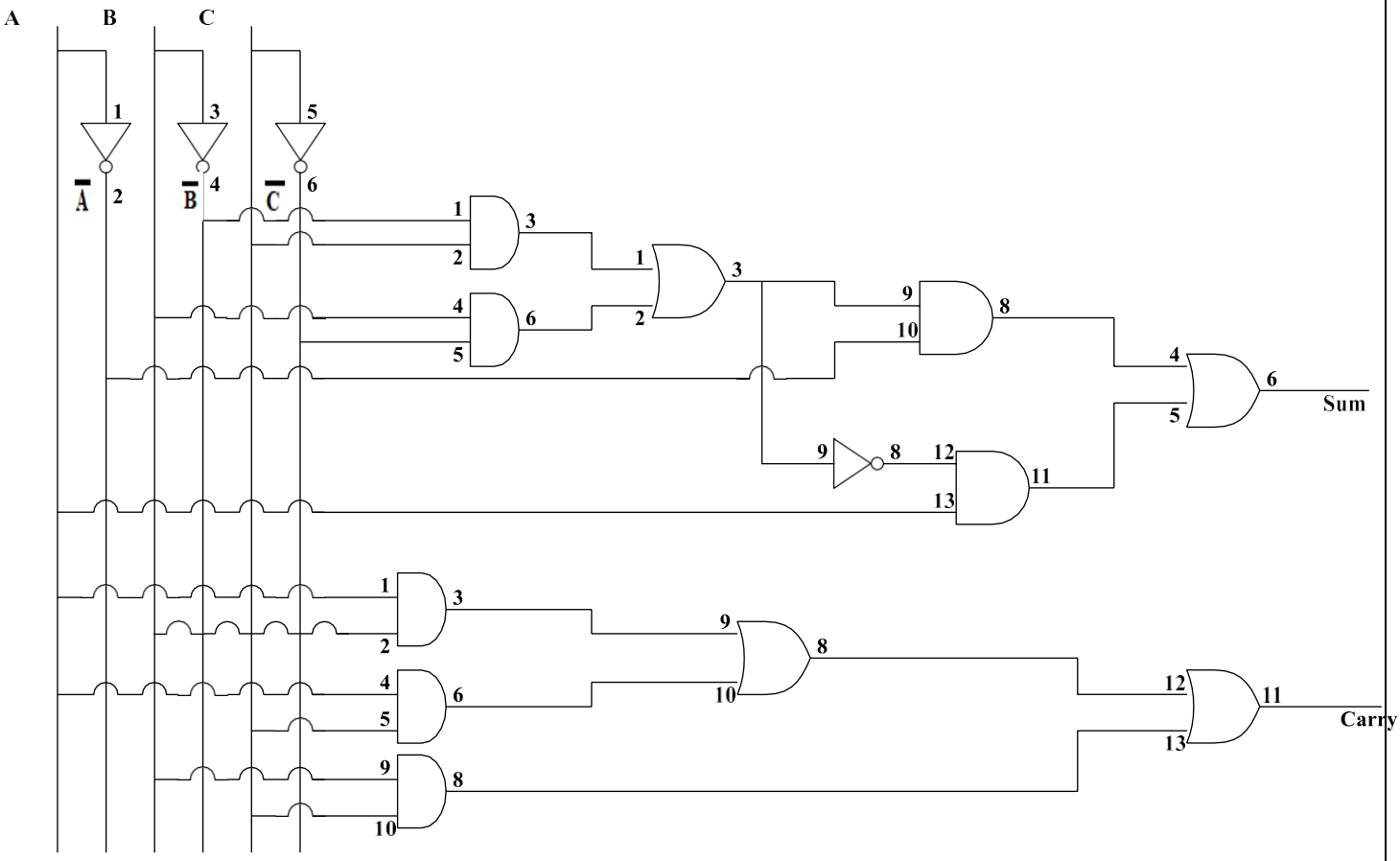
$$Sum = \bar{A}(B\bar{C} + \bar{B}C) + A(BC + \bar{B}\bar{C})$$

K – Map for Carry:



$$Carry = BC + AC + AB$$

Circuit Diagram



Result

Half adder and full adder truth tables are verified

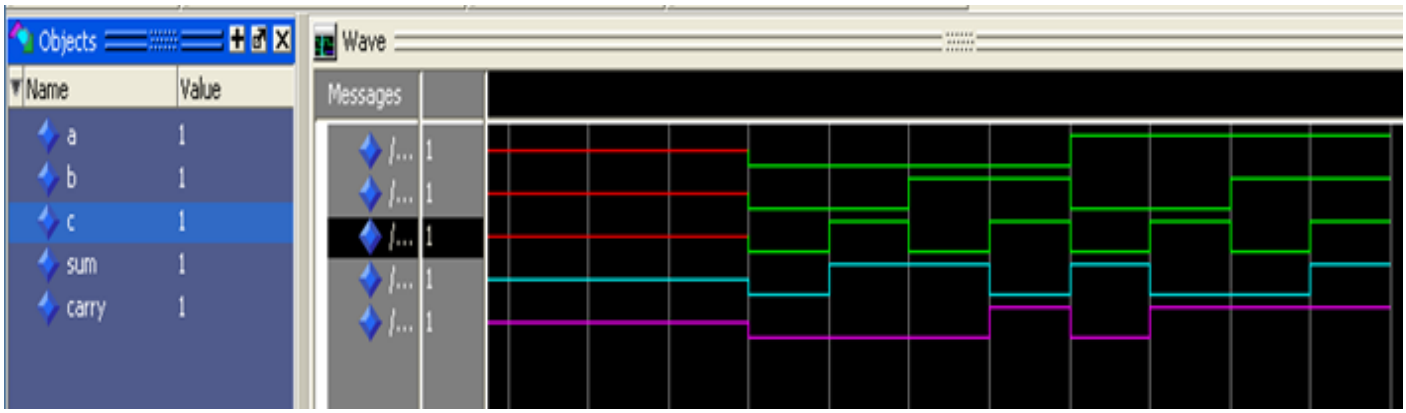
b) Simulate and verify the working of above expressions using VHDL.

Half Adder

```
entity half_Adder is
    Port ( a,b : in STD_LOGIC;
           carry,sum : out  STD_LOGIC);
end half_Adder;
architecture Behavioral of half_Adder is
begin
           sum<=(not(a) and b)or(a and (not(b)));
           carry<=a and b;
     end Behavioral;
```

Full Adder

```
entity FullAdder is
    Port ( a,b,c : in STD_LOGIC;
           sum,carry : out  STD_LOGIC);
end FullAdder;
architecture Behavioral of FullAdder
isbegin
           sum <=(not(c) and (a xor b)) or(c and (a xnor b));
           carry <= ( a and b) or ( a and c) or (b and c);
end Behavioral;
```


Objects
Name
Value
a
1
b
1
c
1
sum
1
carry
1
Wave
Messages

Experiment – 3

a) Given a 4-variable logic expression, simplify it using Entered Variable Map and realize the simplifiedlogic expression using 8:1 multiplexer.

Apparatus Required

SL. No.	Component	Specification
1	8:1 MUX	IC 74151
2	NOT GATE	IC 7404
3	IC TRAINER KIT	-
4	PATCH CORDS	-

Theory

Entered Variable Map: K-map is the best manual technique to solve Boolean equations, but it becomesdifficult to manage when number of variables exceed 5 or 6. So, a technique called Variable Entrant Map (VEM)is used to increase the effective size of k-map. It allows a smaller map to handle large number of variables. Thisis done by writing output in terms of input.

Multiplexers: It is a combinational circuit which have many data inputs and single output depending on control or select inputs. For N input lines, log n (base2) selection lines, or we can say that for 2n input lines, n selection lines are required. Multiplexers are also known as "Data n selector, parallel to serial convertor, many to one circuit, universal logic circuit". Multiplexers are mainly used to increase amount of the data that can be sent over the network within certain amount of time and bandwidth.

Applications

Multiplexers

- Communication System
- Telephone Network
- Computer Memory

Entered Variable Map

- It is commonly used to solve problems involving multiplexers

Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.

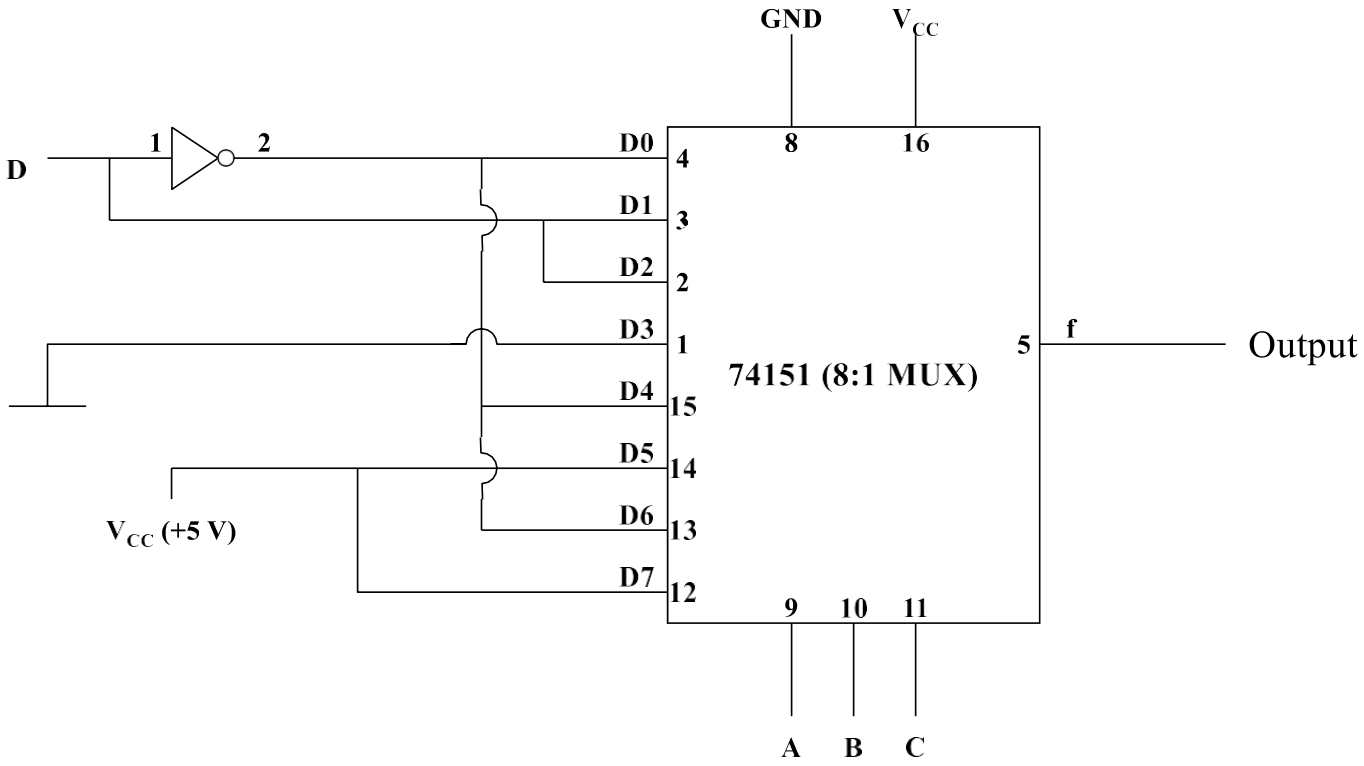
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of thetrainerkit.
5. Switch off the power supply and remove the connections.

F (a, b, c, d) =∑m (0, 3, 5, 8, 10, 11, 12, 14, 15)

Truth Table

Cell No.	a	b	c	d	F(a, b, c, d)	MEV	
0	0	0	0	0	1	$\bar{d}$	D0
1	0	0	0	1	0		
2	0	0	1	0	0	d	D1
3	0	0	1	1	1		
4	0	1	0	0	0	d	D2
5	0	1	0	1	1		
6	0	1	1	0	0	0	D3
7	0	1	1	1	0		
8	1	0	0	0	1	$\bar{d}$	D4
9	1	0	0	1	0		
10	1	0	1	0	1	1	D5
11	1	0	1	1	1		
12	1	1	0	0	1	$\bar{d}$	D6
13	1	1	0	1	0		
14	1	1	1	0	1	1	D7
15	1	1	1	1	1		

Circuit Diagram



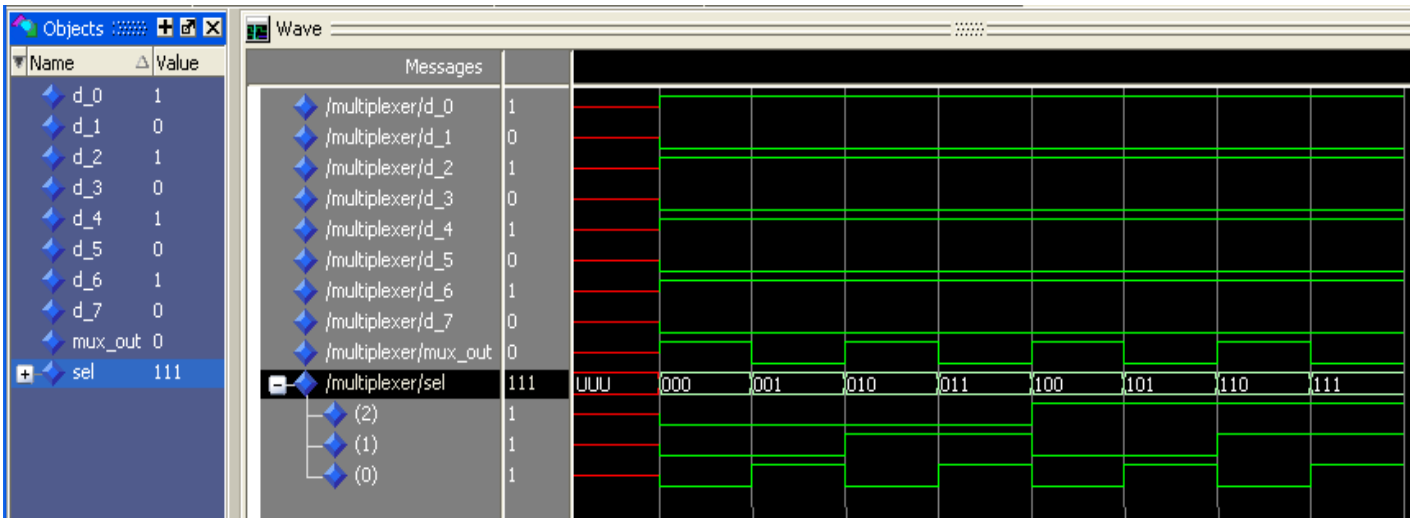
Result:

Given function is verified using the truth table.

b) Simulate and verify the working of 8:1 multiplexer using VHDL.

```
entity Multiplexer is
   Port ( sel : in STD_LOGIC_VECTOR(2 down to 0);
          d_0,d_1,d_2,d_3,d_4,d_5,d_6,d_7: in  STD_LOGIC;
          mux_out : out STD_LOGIC);
end Multiplexer;
architecture Behavioral of Multiplexer is
begin
          process(sel,d_0,d_1,d_2,d_3,d_4,d_5,d_6,d_7)
          begin
                    case sel is
                              when "000"=>mux_out<=d_0;
                              when "001"=>mux_out<=d_1;
                              when "010"=>mux_out<=d_2;
                              when "011"=>mux_out<=d_3;
                              when "100"=>mux_out<=d_4;
                              when "101"=>mux_out<=d_5;
                              when "110"=>mux_out<=d_6;
                              when "111"=>mux_out<=d_7;
                              when others=>null;
                    end   case;
          end process;
          end Behavioral;
```

Select Lines			**Inputs**								**Output**
A	**B**	**C**	**D_0**	**D_1**	**D_2**	**D_3**	**D_4**	**D_5**	**D_6**	**D_7**	
0	0	0	1	0	1	0	1	0	1	0	1
0	0	1	1	0	1	0	1	0	1	0	0
0	1	0	1	0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	1	0	1	0	0
1	0	0	1	0	1	0	1	0	1	0	1
1	0	1	1	0	1	0	1	0	1	0	0
1	1	0	1	0	1	0	1	0	1	0	1
1	1	1	1	0	1	0	1	0	1	0	0


Objects
Name
Value
d_0 1
d_1 0
d_2 1
d_3 0
d_4 1
d_5 0
d_6 1
d_7 0
mux_out 0
sel 111
Wave
Messages
/multiplexer/d_0 1
/multiplexer/d_1 0
/multiplexer/d_2 1
/multiplexer/d_3 0
/multiplexer/d_4 1
/multiplexer/d_5 0
/multiplexer/d_6 1
/multiplexer/d_7 0
/multiplexer/mux_out 0
/multiplexer/sel 111
(2) 1
(1) 1
(0) 1
UUU
000
001
010
011
100
101
110
111

Experiment – 4

a) Design and Implement the Binary to Gray Code converter using Basic Gates.

Apparatus Required

SL. No.	Component	Specification
1	AND GATE	IC 7408
2	OR GATE	IC 7432
3	NOT GATE	IC 7404
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

Binary Number is the default way to store numbers, but in many applications, binary numbers are difficult touse and a variety of binary numbers is needed. This is where Gray codes are very useful.

Gray code has a property that two successive numbers differ in only one bit because of this property gray codedoes the cycling through various states with minimal effort.

Applications

- Analog to digital converters
- Used for error correction in digital communications
- Used in transmission of digital signals

Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of thetrainerkit.

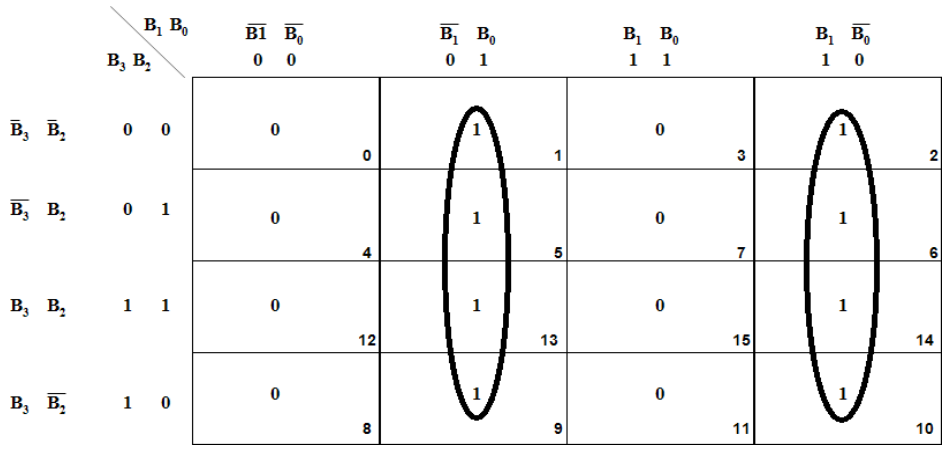
Switch off the power supply and remove the connections.

Truth Table

	Binary Code				Grey Code			
Cell No.	**B_3**	**B_2**	**B_1**	**B_0**	**G_3**	**G_2**	**G_1**	**G_0**
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

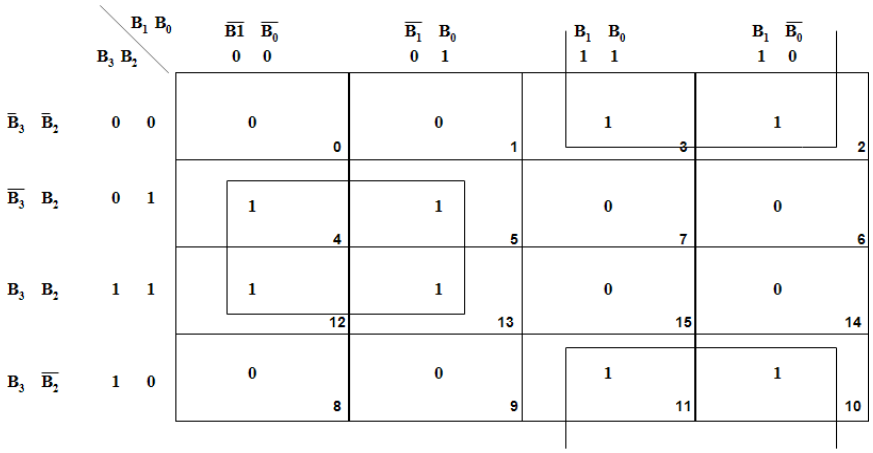
K – Map

G_0



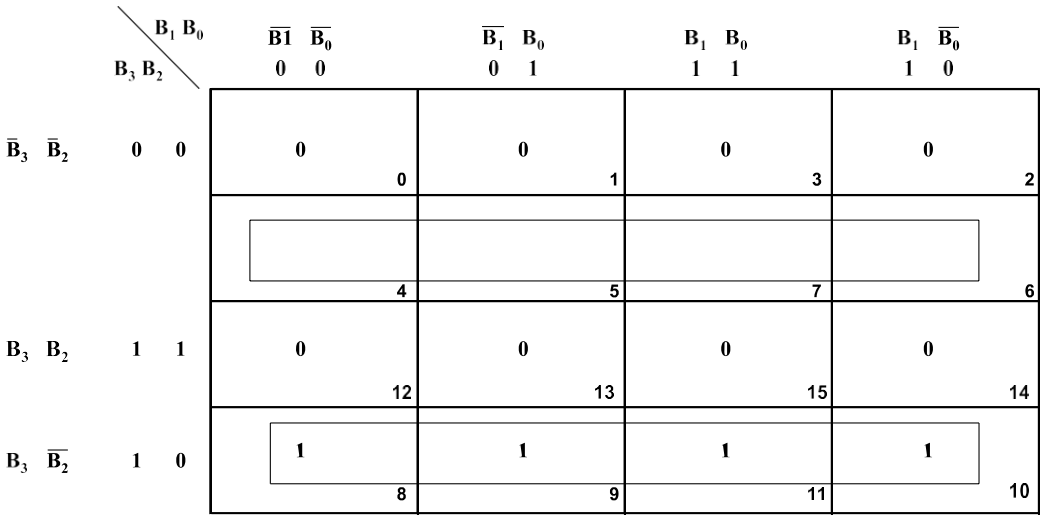
$$G_0 = B0\overline{B}1 + \overline{B}2B1$$

G_1



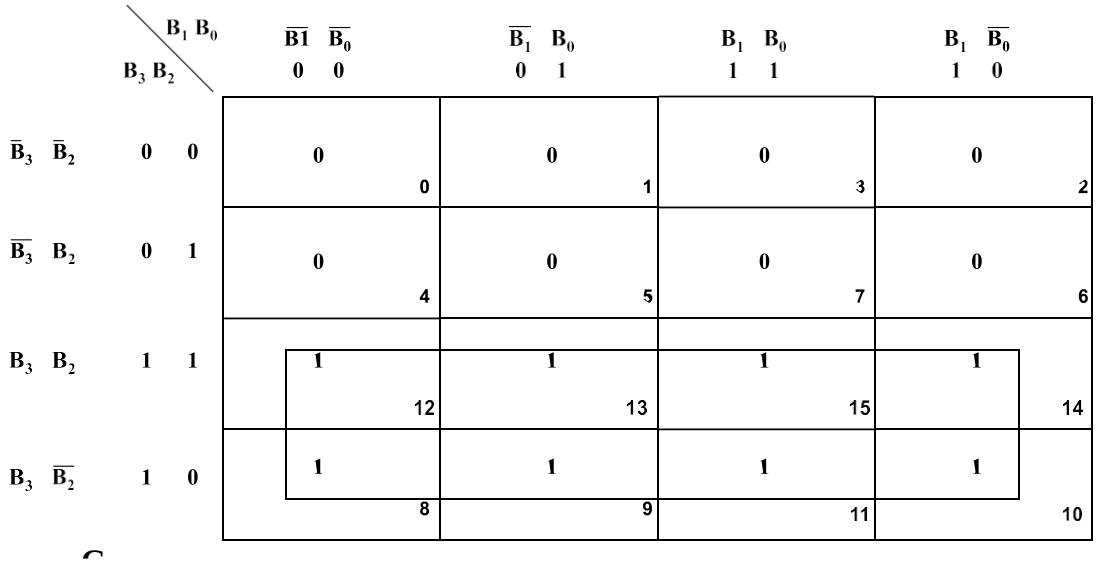
$$G_1 = B2\overline{B}1 + \overline{B}2B1$$

G_2



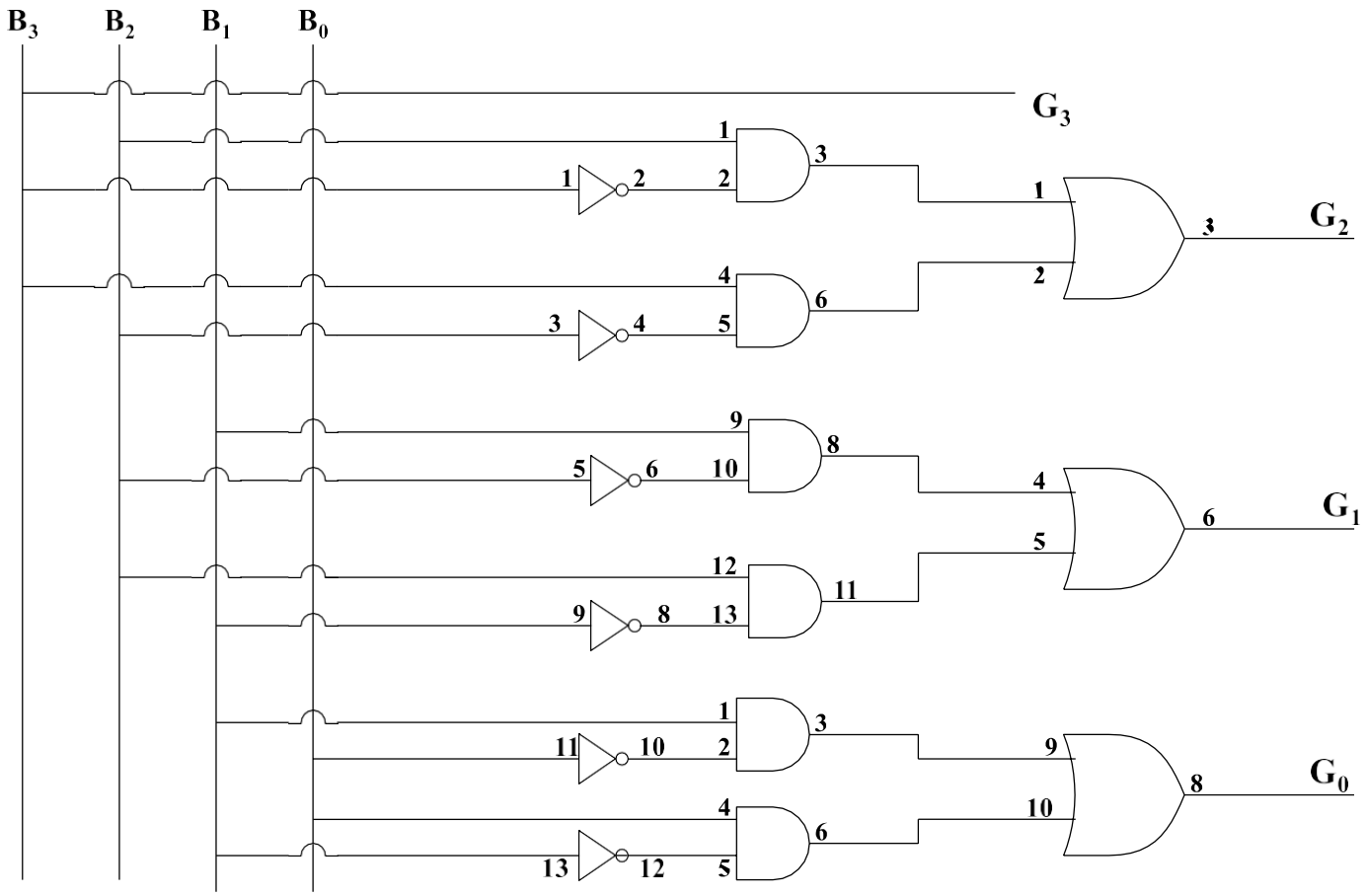
$G_2 = B_2\overline{B_3} + \overline{B_2}B_3$

G_3



G3=B3

Circuit Diagram



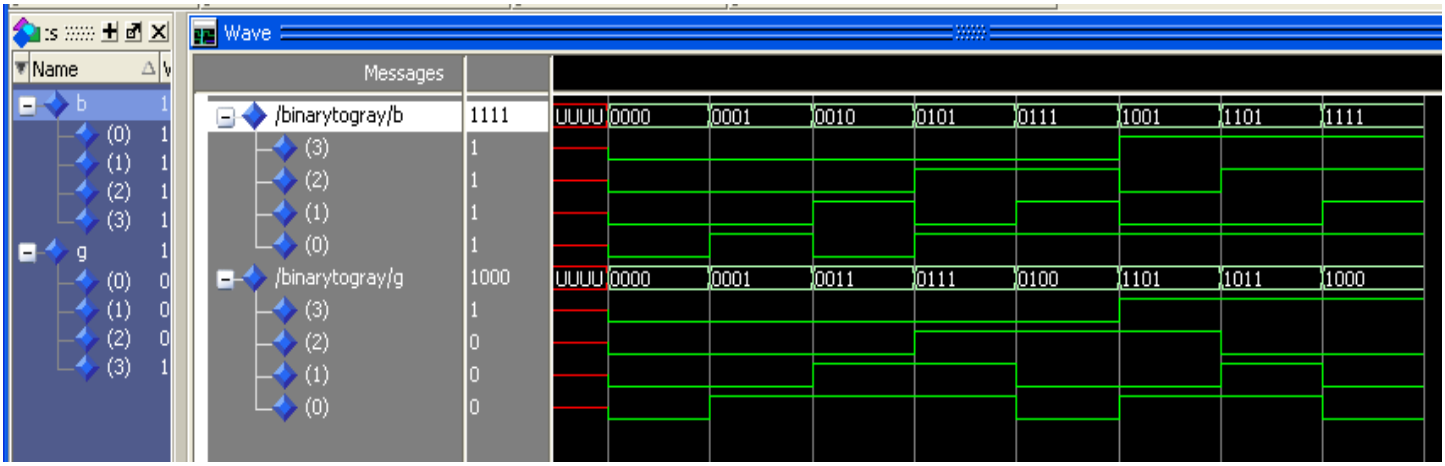
Result:

Binary to Grey code truth table is verified.

b) Simulate and verify the working of Binary to Gray Code converter using VHDL.

```
entity BinaryToGray is
        Port(b : in std_logic_vector(3 downto 0);
        g : out std_logic_vector(3 downto 0));
end BinaryToGray;

architecture Behavioral of BinaryToGray is
begin
        g(3)<= b(3);
        g(2)<= b(3) XOR b(2);
        g(1)<=b(2) XOR b(1);
        g(0)<=b(1) XOR b(0);
end Behavioral;
```



Experiment – 5

Design and Implement the truth table of 3-Bit Parity Generator and 4-bit Parity Checker with an EvenParity bit using basic gates.

Apparatus Required

SL. No.	Component	Specification
1	AND GATE	IC 7408
2	OR GATE	IC 7432
3	NOT GATE	IC 7404
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

Majority of modern communication is Digital in nature i.e., it is a combination of 1's and 0's. The digital data is transmitted either through wires (in case of wired communication) or wireless. Even in an advanced mode of communication, there will be errors while transmitting data (due to noise).

The simplest of errors is corruption of a bit i.e., a 1 may be transmitted as a 0 or vice-versa. To confirmwhether the received data is the intended data or not, we should be able to detect errors at the receiver.

The parity generating technique is one of the most widely used error detection techniques for the data transmission. In digital systems, when binary data is transmitted and processed, data may be subjected to noiseso that such noise can alter 0s (of data bits) to 1s and 1s to 0s.

Hence, a Parity Bit is added to the word containing data in order to make number of 1s either even or odd.

The message containing the data bits along with parity bit is transmitted from transmitter to the receiver.

At the receiving end, the number of 1s in the message is counted and if it doesn't match with the transmitted one, it means there is an error in the data. Thus, the Parity Bit it is used to detect errors, during the transmission of binary data.

A Parity Generator is a combinational logic circuit that generates the parity bit in the transmitter. On theother hand, a circuit that checks the parity in the receiver is called Parity Checker. A combined

circuit or deviceof parity generators and parity checkers are commonly used in digital systems to detect the single bit errors inthe transmitted data.

The sum of the data bits and parity bits can be even or odd. In even parity, the added parity bit will makethe total number of 1s an even number, whereas in odd parity, the added parity bit will make the total numberof 1s an odd number.

Applications

- Most widely used error detection technique for data transmission
- Used in digital systems and many hardware applications
- Used peripheral Component Interconnect (PCI) to detect errors

Procedure

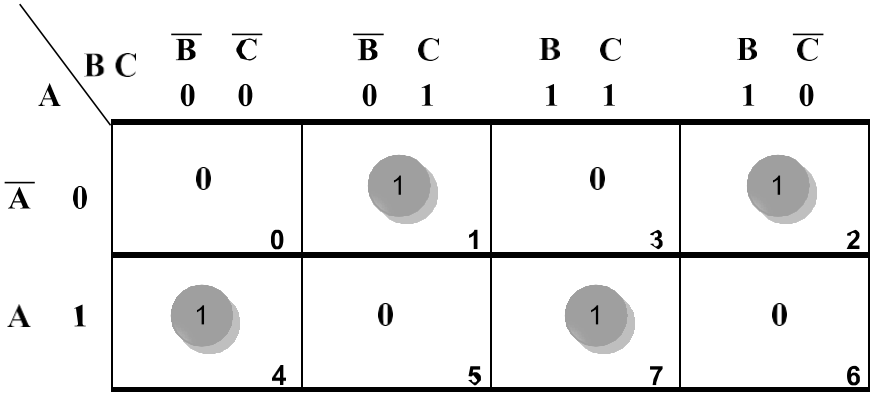
1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of the trainerkit.
5. Switch off the power supply and remove the connections.

3– Bit Parity Generator

Truth Table

Cell. No.	A	B	C	P_{GE}
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	0
6	1	1	0	0
7	1	1	1	1

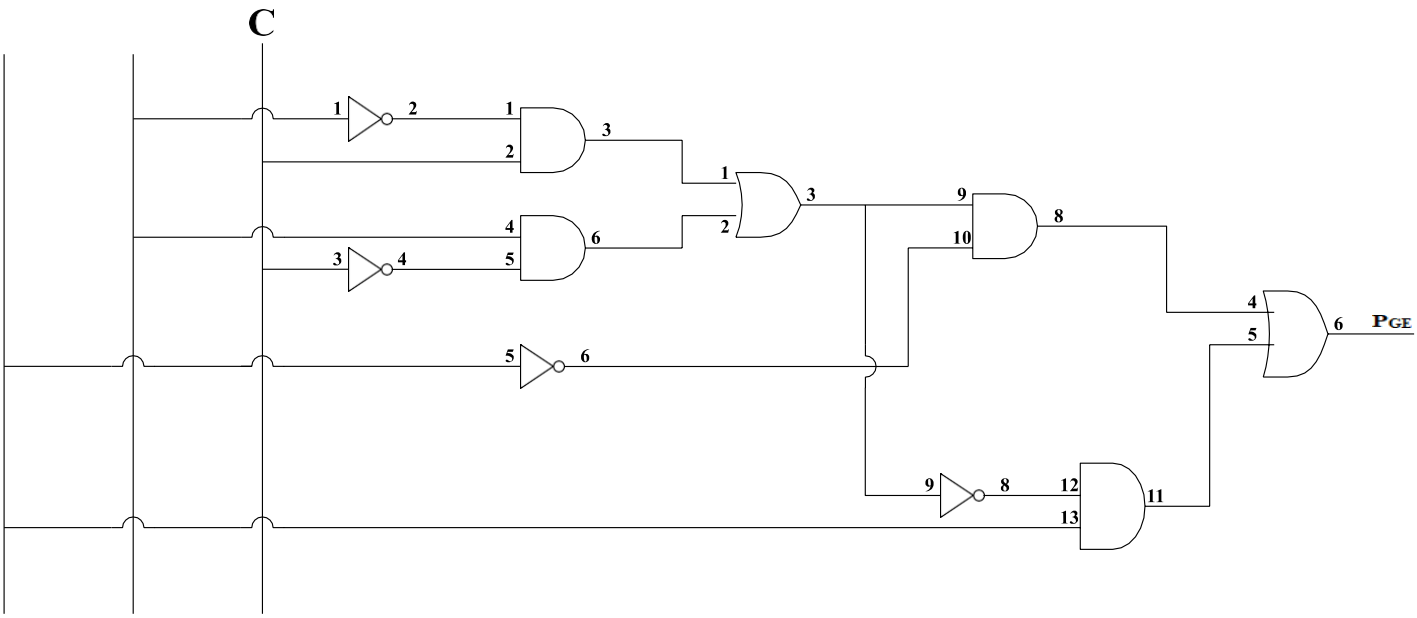
K – Map



$$P_{GE} = \overline{A}BC + \overline{A}B\overline{C} + A\overline{B}C + ABC$$

$$P_{GE} = \overline{A}(\overline{B}C + B\overline{C}) + A(BC + \overline{B}\overline{C})$$

Circuit Diagram



4– Bit Parity Checker

Truth Table

Cell. No.	A	B	C	D	P_{Ch}
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	0

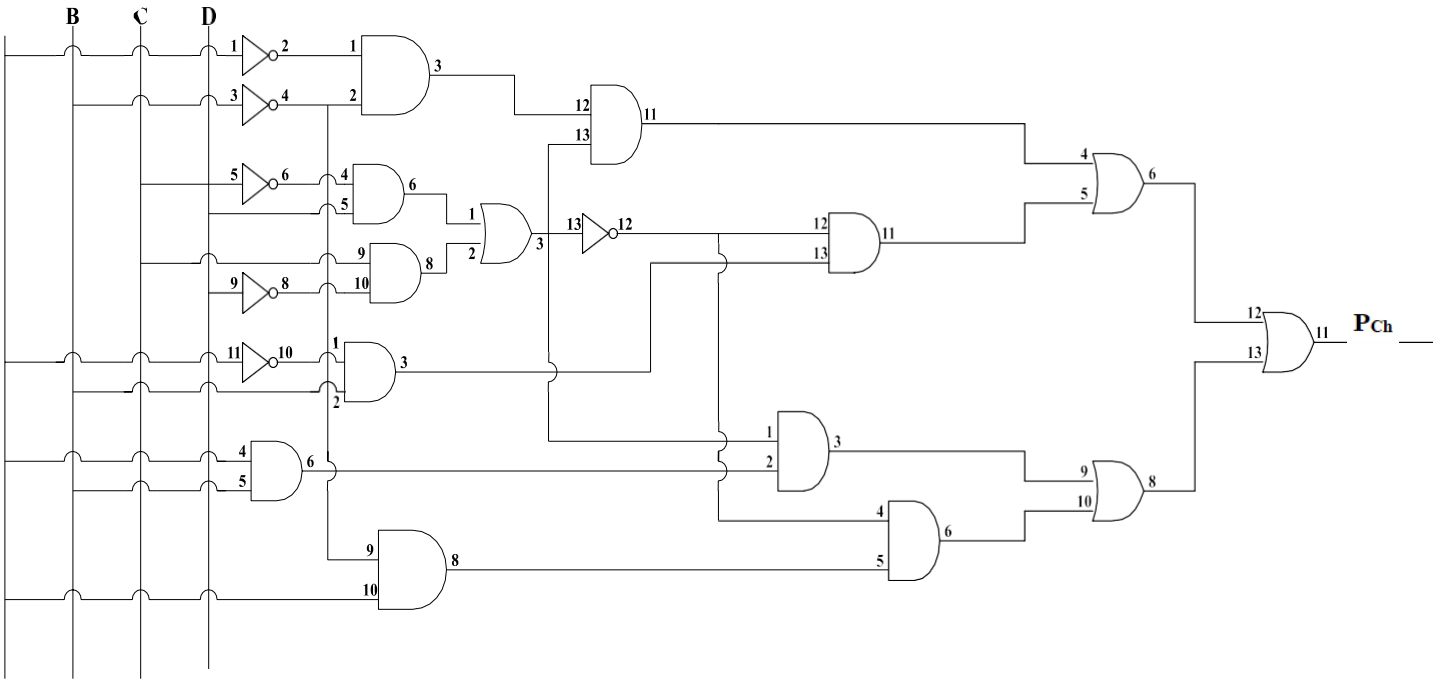
K - Map

AB \ CD	$\bar{C}\,\bar{D}$ (00)	$\bar{C}\,D$ (01)	$C\,D$ (11)	$C\,\bar{D}$ (10)
$\bar{A}\,\bar{B}$ (00)	0 (0)	1 (1)	0 (3)	1 (2)
$\bar{A}\,B$ (01)	1 (4)	0 (5)	1 (7)	0 (6)
$A\,B$ (11)	0 (12)	1 (13)	0 (15)	1 (14)
$A\,\bar{B}$ (10)	1 (8)	0 (9)	1 (11)	0 (10)

PCh $= \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}BCD + AB\bar{C}D + ABC\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}CD$

PCh $= \bar{A}\bar{B}(\bar{C}D + C\bar{D}) + \bar{A}B(CD + \bar{C}\bar{D}) + AB(\bar{C}D + C\bar{D}) + A\bar{B}(CD + \bar{C}\bar{D})$

Circuit Diagram



Result:

Parity generator and checker truth tables are verified.

Experiment – 6

a) Realize a J-K Master / Slave Flip-Flop using NAND gates and verify its truth table.

Apparatus Required

SL. No.	Component	Specification
1	NAND GATE (3 i/p)	IC 7410
2	NAND GATE (2 i/p)	IC 7400
3	IC TRAINER KIT	-
4	PATCH CORDS	-

Theory

Race Around Condition In JK Flip-flop – For J-K flip-flop, if J=K=1, and if clk=1 for a long period of time,then Q output will toggle as long as CLK is high, which makes the output of the flip-flop unstable or uncertain.This problem is called race around condition in J-K flip-flop. This problem (Race Around Condition) can beavoided by ensuring that the clock input is at logic "1" only for a very short time. This introduced the conceptof Master Slave JK flip flop.

The Master-Slave Flip-Flop is basically a combination of two JK flip-flops connected together in a series configuration. Out of these, one acts as the "master" and the other as a "slave". The output from the master flip flop is connected to the two inputs of the slave flip flop whose output is fed back to inputs of the masterflip flop.

In addition to these two flip-flops, the circuit also includes an inverter. The inverter is connected to clock pulse in such a way that the inverted clock pulse is given to the slave flip-flop. In other words if CP=0 for a master flip-flop, then CP=1 for a slave flip-flop and if CP=1 for master flip flop then it becomes 0 for slave flip flop.

Applications

- Registers
- Counters
- Event detectors

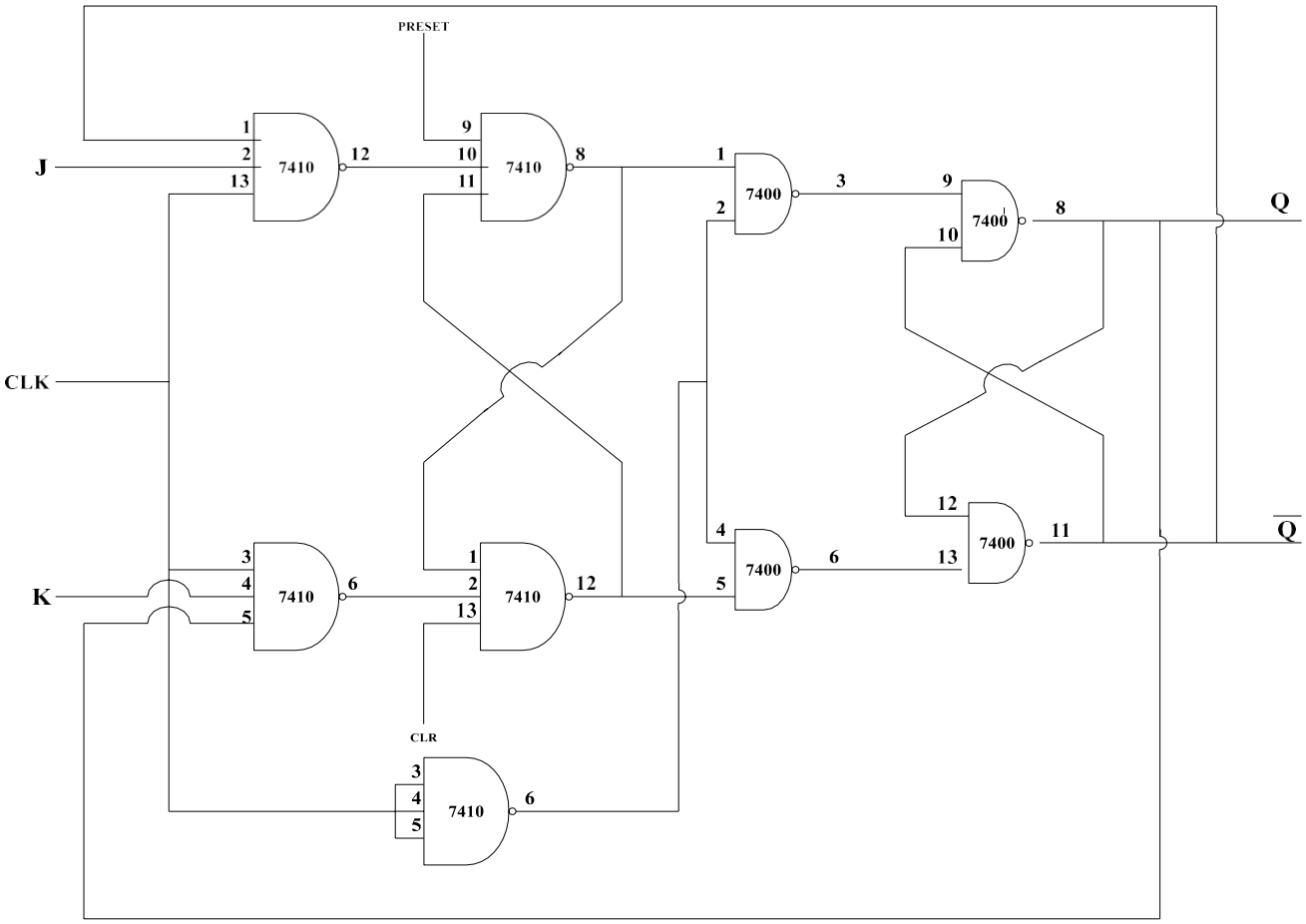
Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of

the trainerkit.

5. Switch off the power supply and remove the connections.

Circuit Diagram



Truth Table

Clock	J	K	Q_{n+1}	$\overline{Q_{n+1}}$	Comments
0	X	X	Q_n	$\overline{Q_n}$	No Change
Pulse	0	0	Q_n	$\overline{Q_n}$	No Change
Pulse	0	1	0	1	Reset
Pulse	1	0	1	0	Set
Pulse	1	1	$\overline{Q_n}$	Q_n	Toggle

Result

JK Master slave flip flop truth table is verified

b) Simulate and verify the working of D Flip-Flop with positive edge triggering using VHDL.

```
entity D_FlipFlop is
    Port ( clk,d : in  STD_LOGIC;
           q : out STD_LOGIC);
end D_FlipFlop;

architecture Behavioral of D_FlipFlop is
begin
process(clk)
begin
          if(rising_edge (clk))then
          q<=d;
          end if;
end process;
end Behavioral;
```

Experiment – 7

a) Design and implement a MOD-n (<8) Synchronous Up Counter using J – K Flip Flop.

Apparatus Required

SL. No.	Component	Specification
1	J K Flip Flop	IC 7476
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

The counters which use clock signal to change their transition are called “Synchronous counters”. This meansthe synchronous counters depends on their clock input to change state values. In synchronous counters, all flip flops are connected to the same clock signal and all flip flops will trigger at the same time.

The result of this synchronization is that all the individual output bits changing state at exactly the same timein response to the common clock signal with no ripple effect and therefore, no propagation delay.

Applications

- Alarm Clock, Set AC Timer, Set time in camera to take the picture, flashing light indicator in automobiles,car parking control etc.
- Mostly used in digital clocks and multiplexing circuits.
- It is also used in digital to analog converters.

Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC’s in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of the trainerkit.
5. Switch off the power supply and remove the connections.

Excitation Table

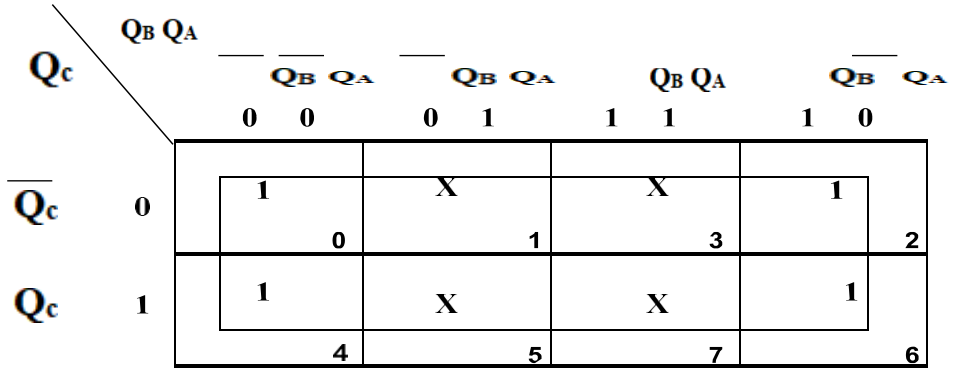
Present State (Q_n)	Next State (Q_{n+1})	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0
X	X	X	X

State Table

Present State			Next State			Flip Flop Inputs					
Q_C	Q_B	Q_A	Q_{C+1}	Q_{B+1}	Q_{A+1}	J_C	K_C	J_B	K_B	J_A	K_A
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

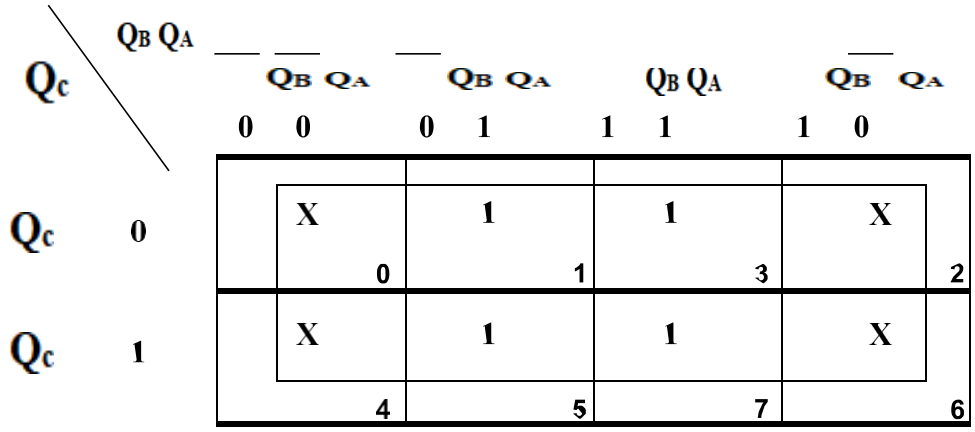
K – Map

J_A



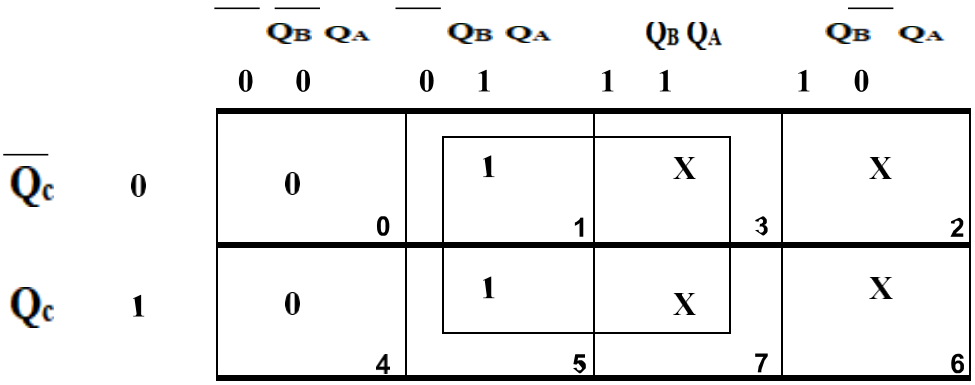
$J_A = 1$

K_A



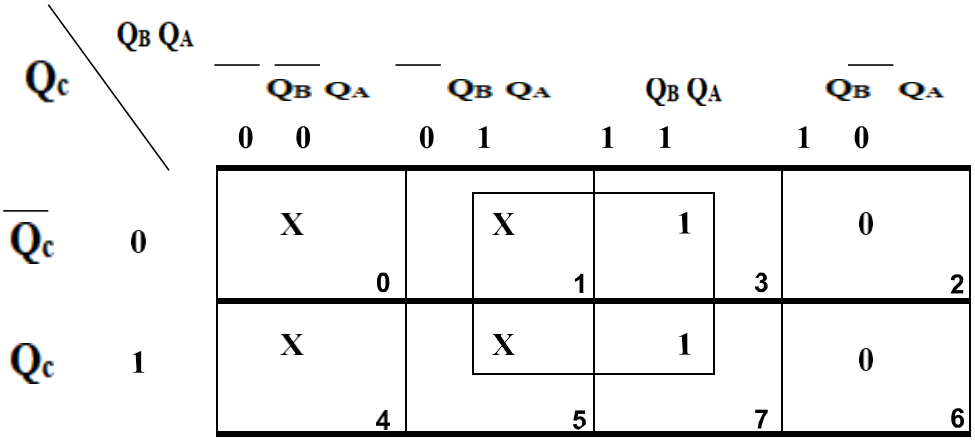
$K_A = 1$

J_B



$J_B = Q_A$

K_B



$K_B = Q_A$

J_C

		$\overline{Q_B}\,\overline{Q_A}$ (0 0)	$\overline{Q_B}\,Q_A$ (0 1)	$Q_B\,Q_A$ (1 1)	$Q_B\,\overline{Q_A}$ (1 0)
$\overline{Q_C}$	0	0 (0)	0 (1)	1 (3)	0 (2)
Q_C	1	X (4)	X (5)	X (7)	X (6)

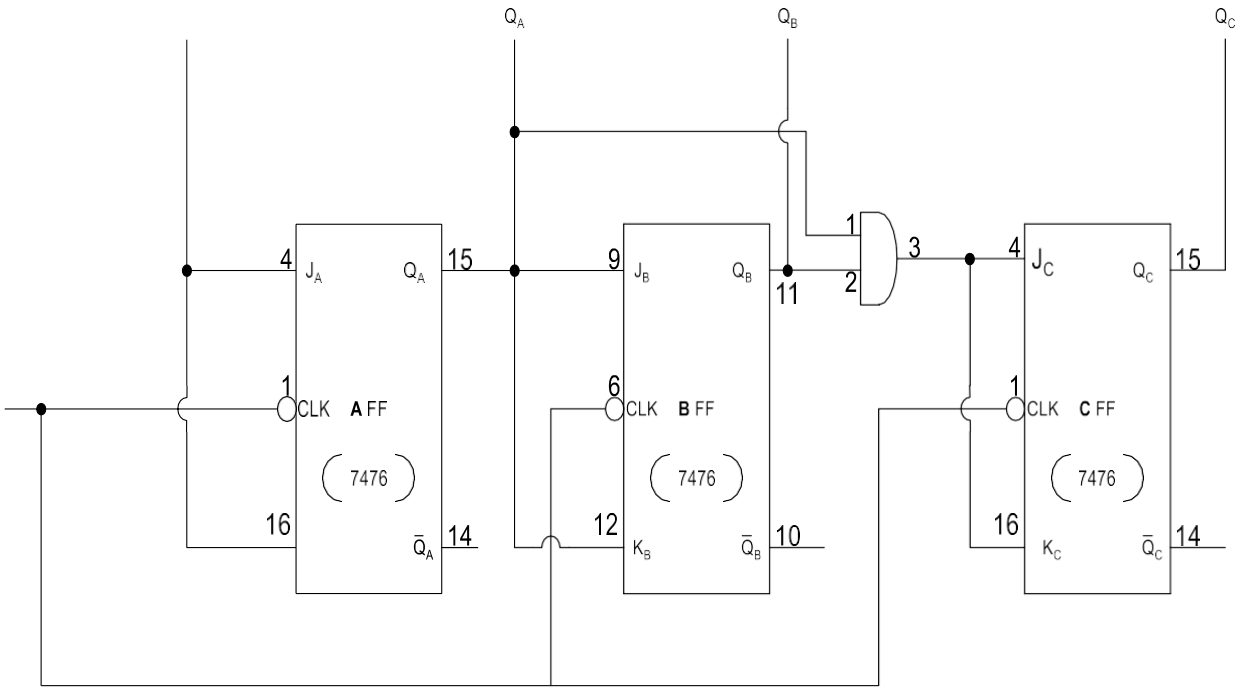
$J_C = Q_B\,Q_A$

K_C

Q_C \ $Q_B\,Q_A$	$\overline{Q_B}\,\overline{Q_A}$ (0 0)	$\overline{Q_B}\,Q_A$ (0 1)	$Q_B\,Q_A$ (1 1)	$Q_B\,\overline{Q_A}$ (1 0)
$\overline{Q_C}$ 0	X (0)	X (1)	X (3)	X (2)
Q_C 1	0 (4)	0 (5)	1 (7)	0 (6)

$K_C = Q_B\,Q_A$

Circuit Diagram



7476 – 5 (V_{CC}), 13 – Gnd

1st IC Pin numbers 2, 3, 7, 8 should be connected to +V_{CC} (5 V)2nd IC Pin numbers 2, 3 should be connected to +V_{CC} (5V)

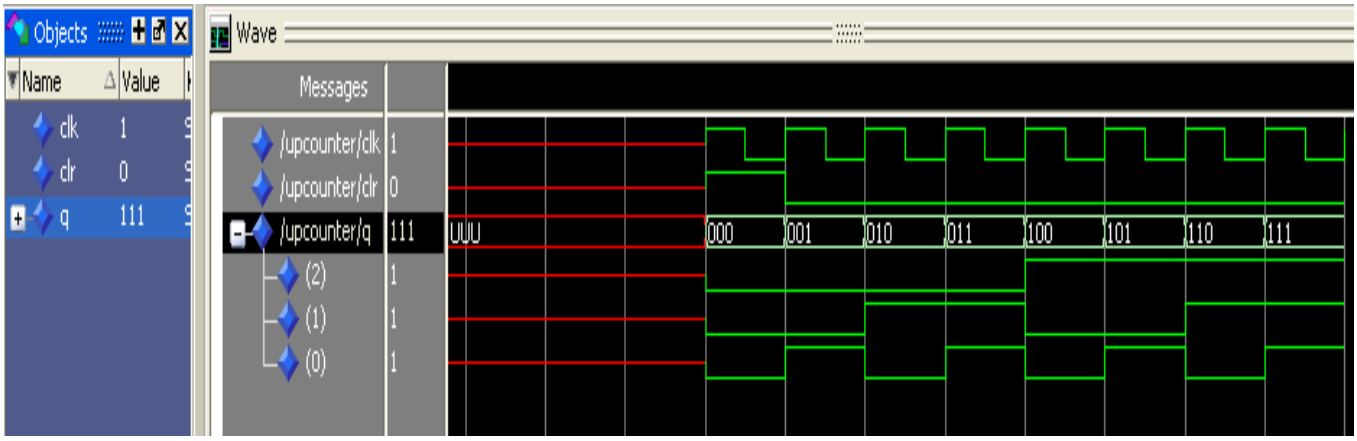
Result

Synchronous counter truth table is verified.

b) Simulate and verify the working of mod-8 up counter using VHDL.

```
entity upCounter is
    Port ( clk : in  STD_LOGIC;
           q : buffer STD_LOGIC_VECTOR(2 downto 0):=”000”);
end upCounter;

architecture Behavioral of upCounter is
begin
process(clk)
begin
        if(clk'event and clk='1')then
                q <= q+1;
        end if;
end process;
end Behavioral;
```



Experiment – 8

a) Design and Implement an Asynchronous Counter using Decade counter IC to Count Up from 0 to n(n<=9) and Demonstrate on 7-Segment Display (using IC7447).

Apparatus Required

SL. No.	Component	Specification
1	BCD – 7 SEGMENT	IC 7447
4	IC TRAINER KIT	-
5	PATCH CORDS	-

Theory

Asynchronous stands for the absence of synchronization. Something that is not existing or occurring at the same time. In asynchronous/ripple counter output of the first flip-flop is provided as the clock to the second flip-flop i.e., flip-flop (FF) are not clocked simultaneously. Circuit is simpler, but speed is slow.

An Asynchronous counter can count using **Asynchronous clock input**. Counters can be easily made using flip-flops. As the count depends on the clock signal, in case of an Asynchronous counter, changing statebits are provided as the clock signal to the subsequent flip-flops. Those Flip-flops are serially connected together, and the clock pulse ripples through the counter. Due to the ripple clock pulse, it's often called a ripple counter. An Asynchronous counter can count $\mathbf{2^n - 1}$ possible counting states.

Applications

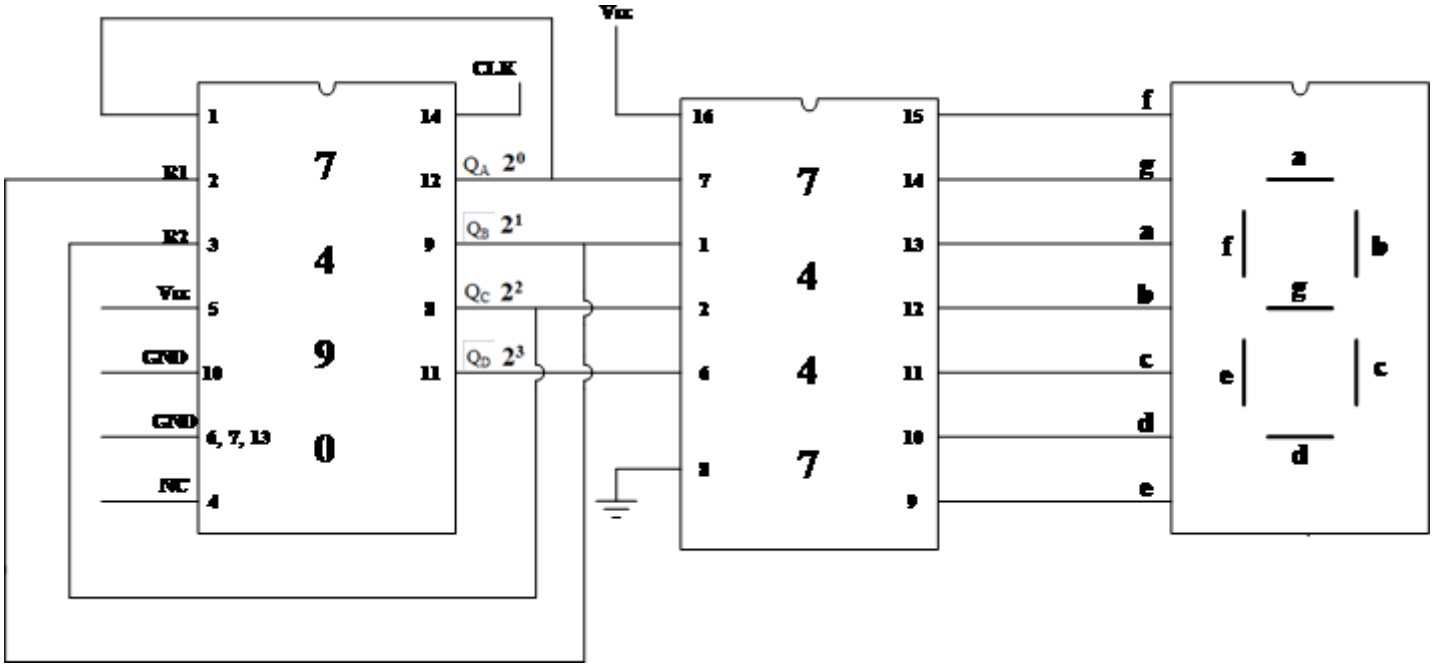
- Asynchronous counters are used as frequency dividers
- Used for low power applications and low noise emission
- Used in Ring and Johnson counter

Procedure

1. Make sure that the given components are working properly before connecting to the trainer circuit.
2. Place the given IC's in the sockets provided on the trainer circuit.
3. Make the connections as per the circuit diagram. Now on the power supply.
4. For the different combinations of the inputs given in the truth table, verify the outputs of the trainerkit.
5. Switch off the power supply and remove the connections.

Circuit Diagram

Mod – 6



Truth Table

Decimal No.	A	b	c	d	e	f	g
0	H	H	H	H	H	H	L
1	L	H	H	L	L	L	L
2	H	H	L	H	H	L	H
3	H	H	H	H	L	L	H
4	L	H	H	L	L	H	H
5	H	L	H	H	L	H	H

Result

Asynchronous counter truth table is verified

a) Simulate and verify the working of Switched tail Counter using VHDL.

```
entity JohnsonCounter is

   Port ( clk : in  STD_LOGIC;
          count: buffer  STD_LOGIC_VECTOR(0 to 3):="0000");
end JohnsonCounter;

architecture Behavioral of JohnsonCounter
isbegin
process(clk)
 begin
 if(clk'event and clk='1')then
       count(0)<= not (count(3));
            for i in 0 to 2 loop
                 count(i+1)<= count(i);
            end loop;
 end if;
end process;
end Behavioral;
```